

Enterprise CI/CD (GitHub Actions & Jenkins)

Category: **Continuous Integration & Continuous Delivery** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Enterprise CI/CD (GitHub Actions & Jenkins)**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

• **OpenID Connect (OIDC) Keyless Cloud Authentication:** GitHub Actions issues short-lived cryptographic JWT tokens that AWS/GCP/Azure accept to assume temporary IAM roles, completely eliminating long-lived static access keys.

Production Application: Protects against catastrophic credential leaks; even if a CI pipeline is compromised, no permanent cloud credentials can be exfiltrated.

• **Matrix Execution Strategy & Build Parallelism:** Configuring build matrices to run test suites across multiple dimensions (operating systems, Node/Python versions, database engines) concurrently.

Production Application: Dramatically reduces cycle times by running comprehensive multi-platform compatibility tests simultaneously in minutes rather than hours.

• **Dependency Caching & Build Acceleration:** Persisting downloaded package caches (`node\_modules`, pip wheels, Maven `.m2`) across workflow runs based on deterministic lockfile hashes.

Production Application: Cuts CI execution times by up to 70%, saving computing costs and accelerating developer feedback loops.

• **Jenkins Master-Agent Architecture & Ephemeral Pods:** Decoupling pipeline orchestration (Jenkins controller) from build execution (ephemeral worker agents spawned dynamically in Kubernetes clusters).

Production Application: Guarantees clean, isolated build environments for every single job while automatically scaling agent capacity to zero when idle.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
<code>gh workflow run deploy.yml -f environment=production -f version=v2.4.1 gh workflow run [options]</code>	Trigger a manual workflow dispatch in GitHub Actions passing parameterized environment and release version inputs. Options: [{flag: '-f', 'description': 'Pass custom input parameter to the workflow.'}, {flag: '--ref', 'description': 'Target Git branch or tag for workflow execution.'}]	Execute during release cutovers to initiate automated canary deployments. Pitfall: Triggering against an unverified feature branch instead of the audited release tag.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
gh run view --log-failed gh run view [options]	Fetch and display only the error and failure log output from a failed GitHub Actions workflow run. Options: [{ 'flag': '--log-failed', 'description': 'Filter and display log lines specifically from failed steps.' }, { 'flag': '--job ', 'description': 'Scope log retrieval to a specific job within the workflow.' }]	Run during pull request triage to immediately isolate which unit test or linter failed. Pitfall: Attempting to view logs of currently active runs; use <code>`gh run watch`</code> to stream live output.
cosign sign --key awskms://arn:aws:kms:us-east-1:123456789:key/abc ghcr.io/org/app:v1.0.0 cosign sign [options]	Cryptographically sign a container image stored in a container registry using a cloud KMS key to verify supply chain integrity. Options: [{ 'flag': '--key ', 'description': 'Reference to cloud KMS key (AWS KMS, GCP KMS, Vault) or local private key.' }, { 'flag': '-a ', 'description': 'Add custom metadata annotations (e.g. Git commit SHA, build number).' }]	Execute immediately after building and scanning a container image in CI before promoting to production. Pitfall: Signing images that have not completed automated vulnerability scans.
trivy image --severity HIGH,CRITICAL --exit-code 1 ghcr.io/org/payment-service:v2.4.1 trivy image [options]	Scan a container image for vulnerabilities, returning non-zero exit code to break the CI pipeline if High or Critical CVEs are detected. Options: [{ 'flag': '--severity ', 'description': 'Filter findings by severity (UNKNOWN, LOW, MEDIUM, HIGH, CRITICAL).' }, { 'flag': '--exit-code 1', 'description': 'Return exit code 1 if matching vulnerabilities are found, halting CI build.' }, { 'flag': '--ignore-unfixed', 'description': 'Ignore vulnerabilities that do not have an upstream patch available.' }]	Run on newly built container images before pushing to production registries. Pitfall: Failing builds on unfixed vulnerabilities without an upstream patch; use <code>`--ignore-unfixed`</code> to focus on actionable fixes.
act -j build -P ubuntu-latest=catthehacker/ubuntu:act-latest act [options] [event]	Simulate and test GitHub Actions workflows locally inside Docker containers without pushing commits to remote repositories. Options: [{ 'flag': '-j ', 'description': 'Execute a specific job defined in the workflow file.' }, { 'flag': '-P ', 'description': 'Map runner label to specific Docker image.' }, { 'flag': '--secret-file ', 'description': 'Inject local mock secrets for testing.' }]	Execute locally before pushing complex workflow changes to GitHub. Pitfall: Expecting cloud OIDC tokens to work locally; mock cloud credentials with local environment variables.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.